MECANIQA API

Catalogo de Pecas e Servicos

OAT 1 - Desenvolvimento Web Orientado a Objetos

Apresentador(es): [NOME COMPLETO DOS INTEGRANTES]
Curso: Sistemas de Informacao
Belo Horizonte, 2026

2. Agenda, problema e objetivo

Objetivo: criar uma API REST para gerenciar pecas e servicos da oficina MecaniQA.

1. Problema e escopo	2. Arquitetura	3. POO	4. API e Singleton	5. Validacao e resultados

Fala sugerida

A oficina precisa organizar suas pecas e servicos. Sem uma estrutura centralizada, consultas de estoque e atualizacoes ficam sujeitas a inconsistencias. O MVP implementa o CRUD completo dos dois recursos. CRUD significa Create, Read, Update e Delete: criar, consultar, atualizar e excluir.

Transicao: com esse objetivo definido, estruturamos um MVP com responsabilidades bem separadas.

3. Escopo do MVP e arquitetura

O fluxo de uma requisicao

Cliente HTTP	Controller REST	Validacao do DTO	Repository Singleton	Lista em memoria

Escopo	Restricoes	Tecnologia
CRUD de pecas e servicos; 10 endpoints; codigos e datas gerados pelo servidor.	Sem banco de dados, Spring Data ou @Autowired. Dados desaparecem ao reiniciar.	Java 21, Spring Boot 4.1.0 e Gradle 9.5.1.

Observacao tecnica: os DTOs separam os dados recebidos dos campos internos como codigo e datas, mas o projeto atual usa um DTO por recurso tanto no POST quanto no PUT.

4. Modelagem orientada a objetos

Classes principais e suas responsabilidades

Peca	Servico	CategoriaPeca	Repositories	Controllers
Atributos privados: codigo, barras, fornecedor, estoque, precos, datas, tamanho, cor e categoria.	Codigo, nome, duracao, custo e datas de criacao e atualizacao.	Enum com MOTOR, SUSPENSAO, FREIOS, ELETRICA e ACESSORIOS.	PecaRepository e ServicoRepository mantem ArrayList em memoria.	PecaController e ServicoController publicam as rotas REST.

Conceitos para destacar

- Encapsulamento: atributos privados acessados por getters e setters.
- Construtores: criam objetos; repositories completam codigo e datas gerados pela aplicacao.
- Associacao: controllers acessam repositories, que mantem as entidades.
- static: usado na instancia unica e no metodo getInstance().

Pergunta provavel: por que usar enum? Resposta: ele restringe a categoria a valores conhecidos e evita grafias inconsistentes.

5. API REST, Singleton e demonstracao

Operacao	Pecas	Servicos	Sucesso
Criar	POST /api/pecas	POST /api/servicos	201 Created
Listar	GET /api/pecas	GET /api/servicos	200 OK
Buscar	GET /api/pecas/{codigo}	GET /api/servicos/{codigo}	200 OK
Atualizar	PUT /api/pecas/{codigo}	PUT /api/servicos/{codigo}	200 OK
Excluir	DELETE /api/pecas/{codigo}	DELETE /api/servicos/{codigo}	204 No Content

Singleton: private static final PecaRepository INSTANCE = new PecaRepository(); | private PecaRepository() {} | public static PecaRepository getInstance()

Na demonstracao, crie uma peca, consulte, atualize e exclua. O codigo e gerado por AtomicLong. Validacoes retornam 400; codigo inexistente retorna 404. O codigo atual nao define Location nem possui tratamento global de erros; as mensagens de erro sao simples objetos com o campo erro.

6. Testes, resultados e limitacoes

Estado verificado no repositorio atual

Item	Resultado
Teste automatizado existente	1 teste de contexto Spring Boot
Execucao validada anteriormente	BUILD SUCCESSFUL com Java 21
MockMvc, testes de repository, Postman e UML	Ainda nao presentes neste repositorio

Limitacoes atuais

- Persistencia somente em memoria: reiniciar a aplicacao apaga os registros.
- Validacao manual nos controllers e respostas de erro simples.
- O PUT atualiza todos os campos do DTO recebidos; nao ha regras de campos imutaveis.
- Para producao: banco de dados, autenticacao, observabilidade, testes de endpoints e tratamento global de erros.

Pergunta provavel: a API esta pronta para producao? Resposta: atende ao MVP academico; ainda exige persistencia, seguranca e observabilidade.

7. Conclusao e referencias

O que entregamos

- CRUD de pecas e servicos por API REST.
- Requisitos de POO, Singleton e armazenamento em memoria atendidos.
- Validacoes de entrada e status HTTP coerentes.
- Codigo-fonte versionado no repositorio do projeto.

Fala sugerida

A MecaniQA API entrega o CRUD de pecas e servicos, aplica orientacao a objetos e usa Singleton nos repositories, respeitando o armazenamento em memoria solicitado. O projeto esta funcional para o escopo do MVP e possui caminhos claros para evolucao. Obrigado pela atencao.

Referencias: repositorio GitHub do projeto; documentacao oficial do Java; documentacao oficial do Spring Boot; materiais da OAT 1.

Checklist antes de apresentar: preencher nomes; executar .\gradlew.bat test; iniciar com .\gradlew.bat bootRun; confirmar http://localhost:8080; revisar o limite de 7 minutos.